

Expense Management — Hackathon-ready Repository

This document contains a complete, GitHub-ready repository structure for the Expense Management app discussed earlier. It includes a Node/Express backend (with currency integration via Rest Countries and exchangerate.host), and a React frontend. Everything you need to copy into a repo is here.

Repo layout

```
expense-hackathon/  
├── backend/  
│   ├── package.json  
│   ├── server.js  
│   ├── db.json  
│   ├── uploads/ (created at runtime)  
│   ├── Dockerfile  
│   └── .gitignore  
├── frontend/  
│   ├── package.json  
│   ├── public/  
│   │   └── index.html  
│   └── src/  
│       ├── index.js  
│       └── App.js  
├── README.md  
└── .gitignore
```

Important: Do **not** paste credentials or secrets into the code. This repo uses simple demo auth via `x-user-email` header for quick hackathon iteration. Replace with JWT / OAuth for production.

Top-level `.gitignore`

```
node_modules/  
backend/node_modules/  
frontend/node_modules/  
backend/uploads/
```

```
.env
.DS_Store
```

README.md (top-level)

Expense Management – Hackathon Starter

This repository contains a hackathon-ready starter for an expense management system.

Features

- Admin signup and company creation
- Create users (employee/manager/admin)
- Upload receipts
- Approval rules (admin-configurable)
- Manager approve/reject flow
- Currency list from Rest Countries and conversion using `exchangerate.host`

Tech stack

- Backend: Node.js + Express + lowdb (file-based DB) + multer for uploads
- Frontend: React (barebones, single page)

Quick start (dev)

1. Start backend

```
```bash
cd backend
npm install
npm run dev
```

#### 1. Start frontend

```
cd frontend
npm install
npm start
```

#### 1. Create company + admin (curl)

```
curl -X POST http://localhost:4000/api/company/signup -H "Content-Type: application/json" -d '{"companyName": "HackCo", "adminEmail": "admin@hackco.test", "adminName": "Admin"}'
```

1. Open frontend at <http://localhost:3000>. Use the demo input to set the `x-user-email` header and test flows.

## Docker (optional)

- Backend Dockerfile included — build and run as needed.

## Backend files

backend/package.json

```
{
 "name": "expense-backend",
 "version": "1.0.0",
 "main": "server.js",
 "scripts": {
 "start": "node server.js",
 "dev": "nodemon server.js"
 },
 "dependencies": {
 "axios": "^1.4.0",
 "body-parser": "^1.20.2",
 "cors": "^2.8.5",
 "express": "^4.18.2",
 "lowdb": "^5.0.0",
 "multer": "^1.4.5",
 "nanoid": "^4.0.0"
 },
 "devDependencies": {
 "nodemon": "^2.0.22"
 }
}
```

backend/.gitignore

```
node_modules/
uploads/
db.json
.env
```

backend/server.js

```
// backend/server.js
const express = require('express');
const { Low } = require('lowdb');
const { JSONFile } = require('lowdb/node');
const multer = require('multer');
const cors = require('cors');
const bodyParser = require('body-parser');
const { nanoid } = require('nanoid');
const path = require('path');
const fs = require('fs');
const axios = require('axios');

const app = express();
app.use(cors());
app.use(bodyParser.json());
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));

// lowdb setup (file database)
const file = path.join(__dirname, 'db.json');
const adapter = new JSONFile(file);
const db = new Low(adapter);

// Ensure DB file exists and default structure
(async function initDB() {
 await db.read();
 db.data ||= { companies: [], users: [], expenses: [], approvalRules: [] };
 await db.write();
})();

// Multer for file uploads
const uploadsDir = path.join(__dirname, 'uploads');
if (!fs.existsSync(uploadsDir)) fs.mkdirSync(uploadsDir);
const storage = multer.diskStorage({
 destination: (req, file, cb) => cb(null, uploadsDir),
 filename: (req, file, cb) => cb(null, `${Date.now()}-${file.originalname}`)
```

```

});
const upload = multer({ storage });

// -----
// Currency cache for restcountries and conversion cache
// -----
let countriesCache = null;
let countriesCacheTime = 0;
const COUNTRIES_TTL = 1000 * 60 * 60; // 1 hour
let ratesCache = {}; // { 'USD_EUR': { rate, time } }
const RATES_TTL = 1000 * 60 * 30; // 30 minutes

// -----
// Helper functions
// -----
async function saveDB(){ await db.write(); }
async function findUserByEmail(email) {
 await db.read();
 return db.data.users.find(u => u.email === email);
}
function requireHeaderUser(req, res, next) {
 const email = req.header('x-user-email');
 if (!email) return res.status(401).json({ error: 'Set x-user-email header for demo auth' });
 req.userEmail = email;
 next();
}
async function requireUserRole(req, res, roles=[]) {
 await db.read();
 const user = db.data.users.find(u => u.email === req.userEmail);
 if (!user) return res.status(403).json({ error: 'User not found' });
 if (roles.length && !roles.includes(user.role)) {
 res.status(403).json({ error: 'Insufficient role' });
 throw new Error('role');
 }
 req.user = user;
}

async function getCompanyById(id) { await db.read(); return db.data.companies.find(c => c.id === id); }

async function fetchCountriesAndCurrencies() {
 const now = Date.now();
 if (countriesCache && (now - countriesCacheTime < COUNTRIES_TTL)) return countriesCache;
 const resp = await axios.get('https://restcountries.com/v3.1/all?fields=name,currencies');
 const data = resp.data.map(c => {

```

```

 const name = c.name && c.name.common ? c.name.common : '';
 const currs = c.currencies ? Object.keys(c.currencies) : [];
 return { name, currencyCodes: currs };
 }).filter(item => item.currencyCodes.length > 0);
 countriesCache = data;
 countriesCacheTime = now;
 return data;
}

async function convertToBaseCurrency(amount, fromCurrency, toCurrency) {
 if (!fromCurrency || !toCurrency) return amount;
 if (fromCurrency === toCurrency) return amount;
 const key = `${fromCurrency}_${toCurrency}`;
 const now = Date.now();
 if (ratesCache[key] && (now - ratesCache[key].time < RATES_TTL)) return
amount * ratesCache[key].rate;
 // Use exchangerate.host free API
 try {
 const resp = await axios.get(`https://api.exchangerate.host/convert?from=${
fromCurrency}&to=${toCurrency}`);
 if (resp.data && typeof resp.data.result === 'number') {
 ratesCache[key] = { rate: resp.data.result, time: now };
 return amount * resp.data.result;
 }
 } catch (err) {
 console.error('convert error', err.toString());
 }
 return amount; // fallback
}

// -----
// Routes
// -----

// Company signup
app.post('/api/company/signup', async (req, res) => {
 const { companyName, adminName, adminEmail, password, baseCurrency = 'USD' }
= req.body;
 await db.read();
 if (!companyName || !adminEmail) return res.status(400).json({ error:
'Missing fields' });
 if (db.data.companies.some(c => c.companyName === companyName)) return
res.status(400).json({ error: 'Company exists' });
 const companyId = nanoid(8);
 db.data.companies.push({ id: companyId, companyName, baseCurrency });
 db.data.users.push({ id: nanoid(), name: adminName || adminEmail.split('@')
[0], email: adminEmail, password: password || null, role: 'admin', companyId });
 await saveDB();

```

```

 res.json({ message: 'Company and admin created', companyId });
 });

// Admin: create user
app.post('/api/users', requireHeaderUser, async (req, res) => {
 try {
 await requireUserRole(req, res, ['admin']);
 } catch(e) { return; }
 const { name, email, role, managerEmail } = req.body;
 await db.read();
 if (db.data.users.find(u => u.email === email)) return res.status(400).json({
 error: 'User exists' });
 const user = { id: nanoid(), name, email, role: role || 'employee',
 companyId: req.user.companyId, managerEmail: managerEmail || null };
 db.data.users.push(user);
 await saveDB();
 res.json({ user });
});

// Admin: set approval rule for category
app.post('/api/approval-rules', requireHeaderUser, async (req, res) => {
 try { await requireUserRole(req, res, ['admin']); } catch(e) { return; }
 const { category, approverEmails = [], minApprovalPercent = 100 } = req.body;
 if (!category) return res.status(400).json({ error: 'category required' });
 await db.read();
 const rule = { id: nanoid(), companyId: req.user.companyId, category,
 approverEmails, minApprovalPercent };
 db.data.approvalRules.push(rule);
 await saveDB();
 res.json({ rule });
});

// Get currency list (proxy to restcountries)
app.get('/api/currencies', async (req, res) => {
 try {
 const data = await fetchCountriesAndCurrencies();
 res.json(data);
 } catch (err) {
 console.error('countries fetch err', err.toString());
 res.status(500).json({ error: 'Failed to fetch currencies' });
 }
});

// Employee: submit expense
app.post('/api/expenses', requireHeaderUser, upload.single('receipt'), async
(req, res) => {
 try { await requireUserRole(req, res, ['employee', 'manager', 'admin']); }
 catch(e) { return; }

```

```

const { description, date, category, paidBy, amount, currency } = req.body;
await db.read();
const company = db.data.companies.find(c => c.id === req.user.companyId);
const amountNum = parseFloat(amount) || 0;
const amountInBase = await convertToBaseCurrency(amountNum, currency,
company?.baseCurrency || 'USD');

const expense = {
 id: nanoid(),
 ownerEmail: req.userEmail,
 companyId: req.user.companyId,
 description,
 date: date || new Date().toISOString(),
 category: category || 'misc',
 paidBy,
 amount: amountNum,
 currency: currency || null,
 amountInBase,
 receiptPath: req.file ? `/uploads/${req.file.filename}` : null,
 status: 'submitted',
 approvals: [],
 createdAt: new Date().toISOString()
};
db.data.expenses.push(expense);
await saveDB();
res.json({ expense });
});

// List expenses
app.get('/api/expenses', requireHeaderUser, async (req, res) => {
 try { await requireUserRole(req, res); } catch(e) { return; }
 await db.read();
 const u = req.user;
 if (u.role === 'employee') {
 const own = db.data.expenses.filter(e => e.ownerEmail === u.email);
 return res.json(own);
 }
 const status = req.query.status;
 let results = db.data.expenses.filter(e => e.companyId === u.companyId);
 if (status) results = results.filter(r => r.status === status);
 if (u.role === 'manager') {
 results = results.filter(e => {
 const assigned = db.data.approvalRules.some(r => r.companyId ===
u.companyId && r.approverEmails.includes(u.email) && r.category === e.category);
 const managerAssignedToUser = (db.data.users.find(x => x.email ===
e.ownerEmail)?.managerEmail === u.email);
 return assigned || managerAssignedToUser || e.approvals.some(a =>
a.approverEmail === u.email);
 });
 }
});

```



```

 });
 }
 res.json(results);
});

// Manager: decision
app.post('/api/expenses/:id/decision', requireHeaderUser, async (req, res) => {
 const { decision, note } = req.body;
 if (!['approve', 'reject'].includes(decision)) return res.status(400).json({
 error: 'decision must be approve or reject' });
 try { await requireUserRole(req, res, ['manager', 'admin']); } catch(e) {
 return; }
 const id = req.params.id;
 await db.read();
 const expense = db.data.expenses.find(e => e.id === id);
 if (!expense) return res.status(404).json({ error: 'expense not found' });

 const approverEmail = req.userEmail;
 const rules = db.data.approvalRules.filter(r => r.companyId ===
 req.user.companyId && r.category === expense.category);
 let allowed = false;
 if (req.user.role === 'admin') allowed = true;
 else {
 allowed = rules.some(r => r.approverEmails.includes(approverEmail)) ||
 (db.data.users.find(u => u.email === expense.ownerEmail)?.managerEmail ===
 approverEmail);
 }
 if (!allowed) return res.status(403).json({ error: 'Not authorized to approve
 this expense' });

 const record = { approverEmail, status: decision === 'approve' ? 'approved' :
 'rejected', time: new Date().toISOString(), note: note || null };
 expense.approvals.push(record);

 let appliedRule = rules[0] || null;
 const approvals = expense.approvals.filter(a => a.status ===
 'approved').length;
 const totalApprovers = appliedRule ? appliedRule.approverEmails.length :
 (expense.approvals.length || 1);

 if (expense.approvals.some(a => a.status === 'rejected')) {
 expense.status = 'rejected';
 } else {
 const percent = Math.round((approvals / Math.max(totalApprovers, 1)) * 100);
 const threshold = appliedRule ? appliedRule.minApprovalPercent : 100;
 if (percent >= threshold) expense.status = 'approved';
 else expense.status = 'pending';
 }
}

```

```
 await saveDB();
 res.json({ expense });
 });

// User list (admin only)
app.get('/api/users', requireHeaderUser, async (req, res) => {
 try { await requireUserRole(req, res, ['admin']); } catch(e) { return; }
 await db.read();
 const users = db.data.users.filter(u => u.companyId === req.user.companyId);
 res.json(users);
});

// health
app.get('/api/health', (req, res) => res.json({ ok: true }));

const PORT = process.env.PORT || 4000;
app.listen(PORT, () => console.log(`Expense backend running on ${PORT}`));
```

---

#### backend/Dockerfile

```
FROM node:18-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm install --production
COPY . .
EXPOSE 4000
CMD ["node", "server.js"]
```

---

#### backend/db.json

```
{
 "companies": [],
 "users": [],
 "expenses": [],
 "approvalRules": []
}
```

## Frontend files

The frontend is intentionally minimal. Use CRA or Vite to scaffold if you prefer. The files below are ready to drop into a `create-react-app` project `src/` and `public/index.html`.

frontend/package.json

```
{
 "name": "expense-frontend",
 "version": "1.0.0",
 "private": true,
 "dependencies": {
 "axios": "^1.4.0",
 "react": "^18.2.0",
 "react-dom": "^18.2.0",
 "react-scripts": "5.0.1"
 },
 "scripts": {
 "start": "react-scripts start",
 "build": "react-scripts build"
 }
}
```

frontend/public/index.html

```
<!doctype html>
<html lang="en">
 <head>
 <meta charset="utf-8" />
 <meta name="viewport" content="width=device-width, initial-scale=1" />
 <title>Expense Hackathon</title>
 </head>
 <body>
 <div id="root"></div>
 </body>
</html>
```

frontend/src/index.js

```
import React from 'react';
import { createRoot } from 'react-dom/client';
```

```
import App from './App';
const root = createRoot(document.getElementById('root'));
root.render(<App />);
```

frontend/src/App.js

```
import React, { useEffect, useState } from 'react';
import axios from 'axios';

const API = 'http://localhost:4000/api';

const DemoHeader = ({ setUser }) => {
 const [email, setEmail] = useState('');
 return (
 <div style={{padding:10, borderBottom:'1px solid #ccc'}}>
 <input placeholder="x-user-email for demo" value={email}
onChange={e=>setEmail(e.target.value)} />
 <button onClick={() => setUser(email)}>Set User</button>
 <small style={{marginLeft:10}}>Use admin email created in backend signup
to act as admin</small>
 </div>
);
};

function App() {
 const [userEmail, setUserEmail] = useState('');
 const [expenses, setExpenses] = useState([]);
 const [form, setForm] = useState({ description:'', amount:'',
category:'misc', paidBy:'cash', currency:'' });
 const [users, setUsers] = useState([]);
 const [newUser, setNewUser] = useState({ name:'', email:'',
role:'employee' });
 const [file, setFile] = useState(null);
 const [countries, setCountries] = useState([]);

 useEffect(()=> { if(userEmail) loadInitial(); }, [userEmail]);

 async function loadInitial(){
 await fetchExpenses();
 try {
 const resp = await axios.get(`${API}/currencies`);
 setCountries(resp.data || []);
 } catch(err){ console.warn('currencies fail', err); }
 try {
 const resp = await axios.get(`${API}/users`, { headers: { 'x-user-email':
```

```

userEmail } });
 setUsers(resp.data || []);
 } catch(e) { /* may not be admin */ }
}

async function fetchExpenses(){
 try {
 const resp = await axios.get(`${API}/expenses`, { headers: { 'x-user-email': userEmail } });
 setExpenses(resp.data);
 } catch(err){ console.error(err); }
}

async function submitExpense(e){
 e.preventDefault();
 const fd = new FormData();
 fd.append('description', form.description);
 fd.append('amount', form.amount);
 fd.append('category', form.category);
 fd.append('paidBy', form.paidBy);
 fd.append('currency', form.currency);
 if (file) fd.append('receipt', file);
 try {
 await axios.post(`${API}/expenses`, fd, { headers: { 'x-user-email': userEmail, 'Content-Type': 'multipart/form-data' } });
 setForm({ description:'', amount:'', category:'misc', paidBy:'cash', currency:'' });
 setFile(null);
 fetchExpenses();
 } catch(err){ console.error(err); alert('submit error') }
}

async function createUser(e){
 e.preventDefault();
 try {
 await axios.post(`${API}/users`, newUser, { headers: { 'x-user-email': userEmail } });
 setNewUser({ name:'', email:'', role:'employee' });
 const resp = await axios.get(`${API}/users`, { headers: { 'x-user-email': userEmail } });
 setUsers(resp.data);
 } catch(err){ console.error(err); alert('create user failed') }
}

async function decision(expenseId, decision){
 try {
 await axios.post(`${API}/expenses/${expenseId}/decision`, { decision }, { headers: { 'x-user-email': userEmail } });
 }
}

```

```

 fetchExpenses();
 } catch(err){ console.error(err); alert('decision failed') }
}

return (
 <div style={{padding:20}}>
 <DemoHeader setUser={setUserEmail} />
 <h2>Logged in as: {userEmail || '-'}</h2>

 <section style={{display:'flex', gap:40}}>
 <div style={{flex:1}}>
 <h3>Submit Expense (Employee)</h3>
 <form onSubmit={submitExpense}>
 <div><input placeholder="description" value={form.description}
onChange={e=>setForm({...form,description:e.target.value})} required /></div>
 <div><input placeholder="amount" type="number" value={form.amount}
onChange={e=>setForm({...form,amount:e.target.value})} required /></div>
 <div>
 <select value={form.category}
onChange={e=>setForm({...form,category:e.target.value})}>
 <option value="misc">Misc</option>
 <option value="food">Food</option>
 <option value="travel">Travel</option>
 </select>
 </div>
 <div>
 <select value={form.currency}
onChange={e=>setForm({...form,currency:e.target.value})}>
 <option value="">Select currency</option>
 {countries.map(ct => (
 ct.currencyCodes.map(code => (
 <option key={`_${ct.name}-${code}`} value={code}>{code} 🇸🇰
{ct.name}</option>
))
))}
 </select>
 </div>
 <div>
 <input type="file" onChange={e=>setFile(e.target.files[0])} />
 </div>
 <button type="submit">Submit</button>
 </form>

 <h3 style={{marginTop:20}}>My Expenses / Pending (based on your
role)</h3>
 <table border="1" cellPadding="6">
 <thead><tr><th>Description</th><th>Amount</th><th>Currency</
th><th>Amount in Base</th><th>Category</th><th>Status</th><th>Receipt</

```

```

th><th>Actions</th></tr></thead>
 <tbody>
 {expenses.map(exp=>(
 <tr key={exp.id}>
 <td>{exp.description}</td>
 <td>{exp.amount}</td>
 <td>{exp.currency || '-'}</td>
 <td>{exp.amountInBase ? exp.amountInBase.toFixed(2) : '-'}</
td>
 <td>{exp.category}</td>
 <td>{exp.status}</td>
 <td>{exp.receiptPath ? <a href={`http://localhost:4000$
{exp.receiptPath}`} target="_blank" rel="noreferrer">view : '-'}</td>
 <td>
 <div>
 <button onClick={()=>decision(exp.id, 'approve')}
>Approve</button>
 <button onClick={()=>decision(exp.id, 'reject')}>Reject</
button>
 </div>
 </td>
 </tr>
)})
 </tbody>
</table>
</div>

<div style={{width:350}}>
 <h3>Admin: Create User</h3>
 <form onSubmit={createUser}>
 <div><input placeholder="name" value={newUser.name}
onChange={e=>setNewUser({...newUser,name:e.target.value})} /></div>
 <div><input placeholder="email" value={newUser.email}
onChange={e=>setNewUser({...newUser,email:e.target.value})} /></div>
 <div>
 <select value={newUser.role}
onChange={e=>setNewUser({...newUser,role:e.target.value})}>
 <option value="employee">Employee</option>
 <option value="manager">Manager</option>
 <option value="admin">Admin</option>
 </select>
 </div>
 <button type="submit">Create</button>
 </form>

 <h3 style={{marginTop:20}}>Existing Users (admin only)</h3>

 {users.map(u => <li key={u.email}>{u.name} ☐ {u.email} ({u.role})</

```

```
li>}}

 </div>
 </section>
</div>
);
}

export default App;
```

---

## What I included

- Backend with currency list proxy ( `/api/currencies` ) that uses restcountries to fetch currency codes and caches for 1 hour.
- Conversion helper using exchangerate.host with caching.
- Frontend currency dropdown that calls `/api/currencies` and stores chosen currency on expense creation.
- Receipt upload and storage in `backend/uploads`.

---

## How to make a GitHub repo and push

```
in project root
git init
git add .
git commit -m "Initial commit - expense hackathon starter"
create repo on GitHub (via UI) then:
git remote add origin git@github.com:YOURUSER/expense-hackathon.git
git push -u origin main
```

---

## Next improvements (pick 1-2 for hackathon)

- Replace demo header auth with JWT + login endpoints.
  - Replace lowdb with Postgres or Mongo and add migrations.
  - Add role-based React UI separate pages and nicer UX (Tailwind/Material UI).
  - Add multi-step approval sequence UI (show who approved, who's next).
  - Add unit tests and a small seed script to populate demo data.
-



Good luck at the hackathon — open the new text doc in the editor and copy files into your local project. If you want, I can now:

- generate a `zip` file here and provide a download link, or
- convert this into a GitHub repo and push it to a repository I create for you (I cannot push to your GitHub; I can provide instructions and a zip).

Which do you want next?